Important Notice

I understand the document is long but bear with me, I have explained each and everything that happened along the way, my thought process, how I overcame with the solutions and why I selected the solution, how this whole project works and every single module I developed using agentic AI. This is not a 100% agentic AI built document, I've put necessary explanation where it was intended.

Kindly give it a look till the end.
Thanks!

Technical Requirements Document (TRD)

Time-Off Microservice

1. Objective

To build a backend microservice which manages the complete lifecycle of timeoff requests made by the employee. These requests will preserve the balance integrity between two systems main HCM and the application that is product facing. HCM will remain the source of truth for the data of timeoff such as balances etc.

Service must be supporting the following functionalities:

- realtime HCM interaction
- batch balance synchronization
- defensive balance validation
- retryable HCM approval sync
- mock HCM endpoints for testing
- NestJS + SQLite implementation
- strong automated test coverage

2. Product Context

Main interface is product facing the employee which will allow them to request their timeoff however the HCM has the authority over the employment and balance data which means that the service must be operational under the following environment:

- employees expect instant feedback and accurate balances
- managers expect approvals to be valid and trustworthy
- HCM balances can change independently outside this service
- Safe handling of synchronization failure

3. Problem Statement

The main challenge in this is to maintain the consistency between a product that faces timeoff workflow and the external record system.

If an employee has 10 days of leave and requests 2 days through the product, the system must do the following:

- validate that the HCM agrees the balance exists
- protect against oversubscription while the request is still pending
- handle temporary HCM failures during approval sync
- reconcile when HCM balances change independently due to processes outside this service

Here an issue raises that the system cannot rely entirely on HCM's error handling due to the fact that HCM-side validation may not always be guaranteed.

4. Goals

- Provide REST endpoints for creating, approving, rejecting, listing, and retrieving time-off requests.
- Maintain local balance snapshots per employee per location.
- Use HCM as the source of truth while still performing balance checks in local environment defensively.
- Support HCM batch balance synchronization so there can be more than one synchronizations at one time.
- Support retryable approval sync through an outbox pattern in case there is an issue with the process somewhere down the line.
- Provide mock HCM endpoints for automated testing.
- Provide rigorous automated tests and proof of coverage.

6. Assumptions

- Balances are maintained per employee per location.
- HCM is the authoritative source of truth meaning that HCM has the authority.
- HCM exposes realtime APIs for validation, balance reads, and apply operations.
- HCM may change balances independently outside this service.
- SQLite is sufficient for this take home scope.
- The product needs fast feedback even though HCM remains authoritative.

7. Design Principles

1. **HCM remains the source of truth** The local service must never attempt to permanently replace HCM authority.

2. **Defensive local integrity** The service should protect against projected oversubscription even if HCM has not yet observed local in flight requests.

3. **Recoverability over brittle synchronous coupling** Temporary HCM failures should not destroy approval intent but what it should do is to keep the process under control while the HCM failure is fixed.

4. **Testability first** Mock HCM behavior must be explicit and controllable through automated tests to check the realtime behaviors in simulated way which will increase the overall security and authenticity of the system.

5. **Minimal complexity for take-home scope** Use the most simple architecture and build a system which is not complex since the scope of this task is not a vast one and doesn't contain any UI, UX, Frontend, Cloud service etc

8. High Level Solution

The architecture I have developed works on request lifecycle management where requests are made and then local snapshots are combined with the request lifecycle, its mentioned below:

- **Time-Off Requests** These will hold lifecycle state, until the approval is confirmed and synced.
- **Balance Snapshots** store the latest known HCM balance plus local reserved balance to check if there are any abnormalities or not because for a developer what's needed is not to just build it system but also to build it in a way that it does not break from basic issues atleast on major part and for the minor part there should be backups available to make patches and update in case if things go south.
- **Projected Available Balance** is calculated locally to prevent oversubscription.
- **Outbox Retry** preserves approval intent when HCM apply fails temporarily.

- **Batch Sync** updates local snapshots when HCM changes independently.
- **Reconciliation** This is an additional backup option as I mentioned above that there should be an option to fix and patch the issues , this will provide an admin/manual repair path to refresh stale snapshots from HCM.

This high level solution will give the managers and employees extremely quick response or feedback while it preserves the correctness in case something fails or if something is consistent.

9. Core Modules

These are the core modules I built using agentic AI. After thinking how the higher level working should be done the next part was to figure out what are the core modules on which whole system will be working.
Their general descriptions and functionalities are mentioned below:

9.1 Time-Off Module

Responsibilities:

- This will allow users to create requests
- validate input and request status transitions
- approve requests
- reject requests
- list and retrieve requests

This is the module that users will be directly interacting if we connect the system to a UI and will give responses to the users as well as act accordingly with other core and extra modules.

9.2 Balances Module

This is passive module who's work is to assess the data in simple terms. It'll do the calculations and return the results to respective module.

Responsibilities:

- store balance snapshots
- calculate reserved balance from active requests
- calculate projected available balance

- ingest batch sync updates
- recalculate snapshot state consistently

9.3 Outbox Module

Once user performs any kind if request then there should be someone who handles the responsibilities of handling the requests in a way that until they are fulfilled it has to keep checks on them.

Responsibilities:

- queue retryable approval sync work
- process pending retry jobs
- complete approval sync after transient HCM failures

9.4 Mock HCM Module

This is just a simulator as the name suggests since we are not connected to a live data and apart from that prototypes need dummy data so this will be working accordingly.

Responsibilities:

- simulate realtime balance validation and application
- simulate external balance changes
- simulate failures such as validation failure, apply failure, and full unavailability

9.5 Reconciliation Module

This is an additional tool.. when I was thinking about the whole problem I had to think that what if everything works smoothly it works right ?
But what if it doesn't?
What if there's a scenario that things hit a bottleneck or what if automated flow breaks ?

The answer was to have a manual administrative control that can take you back to some certain place just like we have a save point or previous working build.
This is that module.

Responsibilities:

- enumerate local snapshots
- pull latest values from HCM
- repair stale local state manually when drift or missed updates are suspected

10. Persistence Model

10.1 TimeOffRequest

Fields:

- id
- employeeId
- locationId
- startDate
- endDate
- days
- status
- managerId
- rejectionReason
- hcmReference
- hcmSyncedAt
- idempotencyKey
- createdAt
- updatedAt

10.2 BalanceSnapshot

Fields:

- employeeId
- locationId
- hcmBalance
- reservedBalance
- projectedAvailableBalance
- balanceDriftDetected
- lastRealtimeCheckedAt

- lastBatchSyncedAt
- lastSyncSource
- createdAt
- updatedAt

10.3 OutboxJob

Fields:

- id
- type
- payload
- status
- attempts
- nextRunAt
- lastError
- createdAt
- updatedAt

11. Request Lifecycle

Here I have explained how the lifecycle of the request will go through.
Document got too long so I just kept it to bullet points to be to the point and to convey my work..
I still this could've been a lot shorter document. But I had to explain everything in case if anyone wants to understand how everything works and wants to continue the development from this time onwards.

11.1 Create Request

1. Validate request input.
2. Reject invalid date ranges.
3. Enforce idempotency using `idempotencyKey`.
4. Validate request with HCM.
5. Fetch realtime HCM balance.
6. Update local realtime snapshot.
7. Check projected available balance locally.
8. Create request as `PENDING_MANAGER_APPROVAL`.

9. Reserve balance locally.

11.2 Approve Request

1. Validate request status.
2. Attempt HCM apply call.
3. If HCM accepts:
    - mark request `APPROVED_SYNCED`
    - release reservation
    - update snapshot from HCM result
4. If HCM rejects:
    - mark request `REJECTED`
    - release reservation
    - persist rejection reason
5. If HCM is temporarily unavailable:
    - mark request `APPROVED_SYNC_PENDING`
    - preserve reservation
    - enqueue outbox retry job

11.3 Reject Request

1. Validate request status.
2. Mark request `REJECTED`.
3. Release reserved balance.

11.4 Outbox Retry

1. Pick pending retry jobs.
2. Retry HCM apply for `APPROVED_SYNC_PENDING` requests.
3. If accepted:
    - mark request `APPROVED_SYNCED`
    - release reservation
    - update snapshot
4. If rejected:
    - mark request `REJECTED`

    - release reservation
5. If temporary failure persists:
    - increment attempts
    - schedule next retry

11.5 Batch Sync

1. Receive a corpus of HCM balances.
2. Recalculate local reserved balances.
3. Update local snapshots from HCM values.
4. Mark reservation-risk drift if known balance is lower than current reservation.

11.6 Reconciliation

1. Load existing balance snapshots.
2. Pull current balances from HCM.
3. Recalculate snapshots from fresh HCM values.
4. Repair stale local state.

12. Consistency Strategy

This design intentionally avoids a distributed transaction across the product service and HCM.

Instead it uses:

- realtime validation and reads for normal operations
- local reservation to prevent oversubscription
- retry/outbox for transient HCM apply failures
- batch sync for externally driven HCM changes
- reconciliation for manual/admin repair

This is an eventual consistency model with local safeguards.

13. API Surface

Time-Off Endpoints

- `POST /time-off-requests`
- `GET /time-off-requests`
- `GET /time-off-requests/:id`

- `POST /time-off-requests/:id/approve`
- `POST /time-off-requests/:id/reject`

Balance Endpoints

- `GET /balances/:employeeId/:locationId`
- `POST /balances/batch-sync`

Reconciliation Endpoint

- `POST /reconciliation/run`

Mock HCM Endpoints

- `GET /mock-hcm/balances/:employeeId/:locationId`
- `POST /mock-hcm/validate`
- `POST /mock-hcm/apply`
- `POST /mock-hcm/admin/set-balance`
- `POST /mock-hcm/admin/failure-mode`

14. Error Handling

Create Flow

- invalid date range → `400`
- HCM validation failure → `400`
- HCM unavailable during create → `503`
- insufficient projected balance → `409`

Approve Flow

- invalid status transition → `409`
- HCM reject → request becomes `REJECTED`
- HCM unavailable → request becomes `APPROVED_SYNC_PENDING`

Retry Flow

- accepted on retry → request becomes `APPROVED_SYNCED`
- rejected on retry → request becomes `REJECTED`
- repeated transient failures → attempts increment and retry is rescheduled

15. Alternatives Considered

Option A — Always read/write directly to HCM with no local snapshot

Rejected. This would preserve strict dependence on the source of truth, but it would provide poor UX, no local defensive projected balance logic, and weaker resilience when HCM is unavailable.

Option B — Make the local service the temporary source of truth

Rejected. This violates the core constraint that HCM is the source of truth and creates a more dangerous reconciliation problem and we cant have that because HCM is the truth and we can not go beyond that.

Option C — Use an event bus or full messaging infrastructure from the start

Rejected for take home scope. Since we don't have a frontend or other things attached as I mentioned above hence it was not mandatory to add this so I set it to reject it every time in this case to keep the system simple.

Option D — Local snapshot + defensive reservation + retry/outbox + reconciliation

Chosen. This preserves HCM authority while still giving the product system responsiveness, defensive behavior, and recovery mechanisms.

16. Key Considerations and Decisions Made

16.1 REST over GraphQL

REST was chosen because the exercise only required REST or GraphQL, and REST was simpler to implement, test, and explain for this scope.

16.2 Keep Reconciliation as Manual/Admin Recovery

A separate reconciliation endpoint is not explicitly required by the brief, but it is useful because HCM balances can change externally. It was kept as a manual/admin recovery path rather than a primary operational path. It is not 100% yet but its just another end point we can have in the future assuming if we were building a project and this was its module.

16.3 Keep `balanceDriftDetected` Semantics Narrow

The current drift flag was kept as a reservation risk indicator (`known balance < reserved balance`) rather than repurposing it late into a general

mismatch detector. This avoided semantic churn and unnecessary last-minute behavioral changes.
This can be updated but for that again the complexity would increase and this was another hurdle I had to think through but decided to keep it as mentioned in above paragraph so to complete the task at hand and not over complicate it.

16.4 Do Not Over-Engineer the Outbox

The retry mechanism is intentionally lightweight and database backed. More advanced methods are available but it will only increase complexity which is not in this scope as far as I can think so I tried to keep it simple.

17. Hurdles Encountered and Resolved

17.1 Dependency mismatch during setup

TypeORM and Nest TypeORM versions were initially incompatible, which caused runtime and compile issues around `DataSource`. The stack was aligned to a working TypeORM 0.3 compatible setup.
I forgot the version of TypeORM but it was around 0.2.1+ so I had to figure out the issue.

17.2 Mixed service refactor state

Some files contained a mix of old and new dependency patterns, which caused duplicate imports and missing property errors. These were normalized.

17.3 Test harness issues

The e2e harness initially failed due to `supertest` import shape and coverage configuration problems. The test setup was corrected so that e2e runs and coverage runs both execute consistently.

17.4 Duplicate balance snapshot logic

Balance recalculation logic was duplicated across paths. It was centralized into the balance service to reduce inconsistency and improve maintainability.

17.5 Repeated request state mutation logic

Time off service approval and rejection flows repeated similar transition logic. Helper methods were introduced to make transitions clearer and less error-prone.

17.6 Reconciliation semantics confusion

Manual testing showed that reconciliation repaired stale local state correctly, but the `balanceDriftDetected` field did not change because its semantics represent reservation-risk, not generic HCM mismatch. The behavior was kept as-is and documented explicitly.

17.7 Unsupported outbox job test semantics

An early test expected unsupported outbox jobs to fail immediately, but the implementation routes them through generic retry handling. The test was aligned with actual implemented behavior rather than forcing unnecessary backend churn late in the process.

18. Testing Strategy

The exercise explicitly emphasizes rigorous tests and protection against regressions.

The automated end to end suite currently verifies:

- request creation and local reservation
- manager rejection and reservation release
- invalid date range rejection
- invalid dimension-style validation failure on create
- general HCM validation failure on create
- HCM unavailability on create
- successful approval and HCM sync
- invalid double-approval attempt
- retry after temporary HCM apply failure
- rejection during retry when HCM no longer accepts the request
- unsupported outbox job retry behavior
- defensive oversubscription rejection
- batch sync after independent HCM balance changes

- drift detection when batch balance drops below reserved balance
- idempotent request creation
- reconciliation of stale local balance snapshots

Verified automated results:

- 16/16 tests passing
- 93.13% statement coverage
- 56.04% branch coverage
- 90.14% function coverage
- 93.36% line coverage

```
> timeoff-microservice-reference@1.0.0 test:e2e
> jest --config ./test/jest-e2e.json

 PASS  test/timeoff.e2e-spec.ts (11.853 s)
  Time-off microservice (e2e)
    √ creates a request and reserves local balance (192 ms)
    √ rejects a request and releases local reservation (68 ms)
    √ rejects invalid date ranges (13 ms)
    √ rejects request creation for an invalid combination of dimensions (24 ms)
    √ rejects request creation when HCM validation fails (26 ms)
    √ rejects request creation when HCM is unavailable (184 ms)
    √ approves a request and syncs the HCM balance (76 ms)
    √ prevents approving a request that is already approved (65 ms)
    √ queues retry when HCM apply is unavailable and later succeeds (114 ms)
    √ rejects pending-sync requests on retry when HCM no longer accepts them (129 ms)
    √ requeues unsupported outbox job types through generic retry handling (33 ms)
    √ defensively rejects oversubscription even if HCM has not seen local reservations yet (90 ms)
    √ accepts batch sync updates when HCM balance changes independently (16 ms)
    √ flags drift when batch sync balance is lower than reserved balance (57 ms)
    √ returns the same request for the same idempotency key (33 ms)
    √ reconciles stale local balance snapshots with HCM (171 ms)

Test Suites: 1 passed, 1 total
Tests:       16 passed, 16 total
Snapshots:   0 total
Time:        12.49 s
Ran all test suites.
```

Figure 1 test:e2e results

```
PASS  test/timeoff.e2e-spec.ts (14.304 s)
  Time-off microservice (e2e)
    √ creates a request and reserves local balance (196 ms)
    √ rejects a request and releases local reservation (79 ms)
    √ rejects invalid date ranges (13 ms)
    √ rejects request creation for an invalid combination of dimensions (32 ms)
    √ rejects request creation when HCM validation fails (26 ms)
    √ rejects request creation when HCM is unavailable (134 ms)
    √ approves a request and syncs the HCM balance (75 ms)
    √ prevents approving a request that is already approved (58 ms)
    √ queues retry when HCM apply is unavailable and later succeeds (131 ms)
    √ rejects pending-sync requests on retry when HCM no longer accepts them (122 ms)
    √ requeues unsupported outbox job types through generic retry handling (28 ms)
    √ defensively rejects oversubscription even if HCM has not seen local reservations yet (56 ms)
    √ accepts batch sync updates when HCM balance changes independently (16 ms)
    √ flags drift when batch sync balance is lower than reserved balance (60 ms)
    √ returns the same request for the same idempotency key (45 ms)
    √ reconciles stale local balance snapshots with HCM (224 ms)
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	93.13	56.04	90.14	93.36	
balances	97.1	66.66	100	96.82	
balance-snapshot.entity.ts	100	100	100	100	
balances.controller.ts	100	100	100	100	
balances.service.ts	95.12	66.66	100	94.87	26,101
balances/dto	90	100	0	90	
upsert-balance-batch.dto.ts	90	100	0	90	34
common/enums	100	100	100	100	
request-status.enum.ts	100	100	100	100	
config	100	100	100	100	
database.config.ts	100	100	100	100	
hcm	95.65	100	100	95.23	
hcm.types.ts	100	100	100	100	
http-hcm.client.ts	95.45	100	100	95	35
mock-hcm	90.54	73.68	88.23	90	
mock-hcm.controller.ts	95	100	85.71	94.44	39
mock-hcm.service.ts	88.88	73.68	90	88.46	51-54,61,87,113
outbox	97.67	61.11	100	98.76	
outbox-job.entity.ts	100	100	100	100	
outbox.service.ts	97.05	56.25	100	98.46	60
reconciliation	100	100	100	100	
reconciliation.controller.ts	100	100	100	100	
reconciliation.service.ts	100	100	100	100	
time-off	87.95	37.83	84	88.6	
time-off-request.entity.ts	100	100	100	100	
time-off.controller.ts	94.73	100	83.33	94.11	22
time-off.service.ts	85.15	37.83	84.21	86.29	...,216,249,294-305,314,371,406
time-off/dto	100	100	100	100	
create-time-off-request.dto.ts	100	100	100	100	
request-action.dto.ts	100	100	100	100	

```
Test Suites: 1 passed, 1 total
Tests:       16 passed, 16 total
Snapshots:   0 total
Time:        15.705 s
Ran all test suites.
```

Figure 2 test:cov results

19. Reliability Notes

This solution improves robustness by:

- rejecting invalid date ranges early
- rejecting invalid transitions
- enforcing projected balance defense locally
- preserving approval intent during transient HCM failures
- accepting independent HCM batch updates
- supporting manual reconciliation of stale snapshots
- verifying idempotent create behavior

20. Tradeoffs

- SQLite is appropriate for the exercise but would not be the final production database choice.
- Branch coverage is improved significantly but is not exhaustive.
- `balanceDriftDetected` is currently reservation-risk oriented rather than a full external mismatch detector.
- Reconciliation is manual/admin triggered rather than scheduled automatically.

21. Deliverables Alignment

This implementation aligns with the requested deliverables:

- TRD
- code repository
- test cases
- proof of coverage

22. Future Improvements

- As mentioned, add a distinct snapshot mismatch flag separate from reservation risk drift which will need a bit of change In core logic
- add more branch focused tests for deeper edge cases
- improve unsupported outbox job handling semantics if required by future product expectations
- introduce stronger metrics and observability around retries and reconciliation

- add authentication, authorization, and audit logging for production readiness
- consider background reconciliation scheduling if operational needs grow